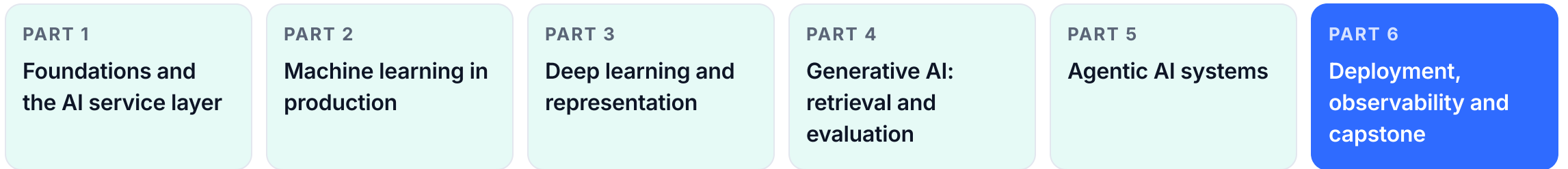


Deployment, observability and capstone

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you put the whole system on your cloud: least-privilege identity, a secret that never touches an image, readiness the platform acts on, a dashboard, a budget alarm and a rollback you have run. Then you measure the cost per resolved ticket, before and after a router.

Where we are



Five parts built four services and the client under them. The capstone deploys the lot and puts Project 1 in front of the agent so the cheapest model that suffices answers each ticket.

Today, step by step

- 1 Build the images and inspect the layers
- 2 Read the cost sheet and the review test
- 3 Route cheap first
- 4 Deploy the Session path
- 5 Prove the identity is least privilege and the image is clean
- 6 Measure cold and warm latency, then roll back
- 7 Capstone: cost per resolved ticket, before and after
- 8 Watch the Reference stack, then decide about yours
- 9 Session check, demos, debrief

Numbers go in one file

Every step that produces a number writes it to `sessions/session-06/NOTES.md` : three cost sheet figures, the first cold start, both p95s, the rollback command, two costs per resolution.

`/session-check 6` reads that file and asks for anything missing.

Two tiers, one set of images

Tier	Who	What it proves
Session path	You, today	The same containers on the track's serverless container runtime, least-privilege identity, readiness, metrics, alarms, a budget, a rehearsed rollback
Reference stack	The instructor, live	The agent on the track's managed agent runtime, native guardrails, and the platform enforcing the escalation gate a second time

Nothing taught on the Session path is thrown away: the Reference stack uses the same images, the same health checks, the same metrics.

One recipe builds every service, and no layer carries a secret

```
make images  
docker history nw-agent --no-trunc | head -30
```

- Two stages, a non-root user, locked dependencies, weights and indexes baked at build and loaded once at startup
- Readiness on `/readyz`, liveness on `/healthz`, a graceful shutdown so in-flight requests finish
- Five images. `nw-agent` is the largest at two to three gigabytes: every artifact and the embedding models

Not in the image

`.env`, `.aws`, the ticket corpus.

`.dockerignore` keeps them out.

The runtime stage receives only the virtualenv, the package, the accounts fixture and the artifacts you named. The check is the history output, not this slide.

Serverless bills nothing idle; the images and one secret do

	AWS Lambda, Session path	GCP Cloud Run, Session path
The 2-hour session	under 0.10 USD, compute inside the 400,000 GB-second free tier	under 0.20 USD, request-based billing while serving
Idle per day	0.03 USD: ECR images and the Secrets Manager secret	0, min instances 0
After <code>make stop</code>	about 0.80 USD per month, reserved concurrency 0	pins each service to one instance
After <code>make destroy</code>	0	0

App Runner is closed to new customers; ECS Express Mode is the always-warm option, and it would add about 1.20 USD per day. Credits to request: 25 USD per participant, assuming you destroy after the session.

Route cheap first: Project 1 scores a ticket for nothing

```
trriage = await registry.execute("classify_urgency", {"subject": subject, "body": body})
priority, p0 = _read_trriage(trriage.content) if triage.ok else ("P2", 0.0)
if priority == "P0" and p0 >= policy.p0_confidence:                # 0.6: no model call
    return escalation_proposal(tier="duty_manager", cost_usd=0.0)
if priority in policy.economy_priorities:                          # ("P3",)
    return await run_agent(role=ModelRole.ECONOMY, max_steps=4, budget_usd=0.05)
return await run_agent(role=ModelRole.WORKHORSE, max_steps=10, budget_usd=0.40)
```

Confident P0

A proposed escalation, cost zero. The loop's approval gate still applies.

P3

The Economy model under small caps.

Everything else

The Workhorse loop from the agent part.

The key lives in a secret, never in the environment or a layer

AWS. CDK generates the API key into Secrets Manager. The function reads it once per cold start. `NW_API_KEY` in the environment prints `None`; `NW_API_KEY_SECRET_ARN` is what the function holds.

GCP. Terraform generates it, stores it in Secret Manager and mounts it into every service as `NW_API_KEY`. `terraform output -raw api_key` is how you read it.

The 401 is always the header

Every route except the probes and `/metrics` answers 401 without `x-api-key`. Someone reads that as an IAM problem and spends ten minutes in the console.

An IAM problem is an access error on the model call, and it says so in the deploy output.

Least privilege has a name on every track

	AWS	GCP
Identity	one execution role per function	one service account per service
Who may call a model	only the policy and agent functions, <code>bedrock:InvokeModel</code> on three model ARNs, never <code>foundation-model/*</code>	only the policy and agent services hold <code>roles/aiplatform.user</code> ; nothing holds Editor
What triage holds	basic execution for logs and X-Ray write, no <code>bedrock:</code> action	trace and metric writer roles only
What refuses a mistake	the synth test refuses a wildcard before anything deploys	<code>terraform plan</code> shows the triage account with no <code>aiplatform</code> binding

Observability is already there, and none of it is self-hosted

Signal	AWS	GCP
Traces	OpenTelemetry spans to the X-Ray OTLP endpoint, searchable in CloudWatch Transaction Search	the same spans through the Cloud Trace exporter
Metrics	Lambda invocation, error and duration; Bedrock tokens and throttles per model; your counters, exported once a minute as an EMF log line	Cloud Run request, latency and error, split by revision; your counters, exported once a minute as a JSON line that log-based metrics read
Dashboard	one CloudWatch dashboard, generated: what Lambda saw next to what the service measured	one Cloud Monitoring dashboard, generated, the same two views
Alarms	errors and p95 latency per function, to SNS; the same two roll a deployment back	5xx and p95 latency per service, plus a service-measured p95 with a bar per service, to email
Drift	metric filter on <code>drift_alert</code> in the triage log group, alarm to SNS	log-based metric on the same line, alert policy to email
Budget	AWS Budgets at 50, 80 and 100 percent	billing budget at 50, 80 and 100 percent

AI systems degrade silently; three panels make it loud

A model answers slower

Nothing errors. Users wait.

p95 latency

A retriever returns nothing

The model answers anyway,
from nothing.

Refusal rate

A judge drifts

The harness keeps passing
tickets it should fail.

Cost per resolution

Those are the panels the dashboard generates. Everything else on it is context for those three.

Always warm buys back the cold start at a price you can name

2.30 USD

per day for one always-warm 8 GB agent function on AWS, about 70 USD a month

- Cold start: the container pulls a multi-gigabyte image and loads the models. Twenty to forty seconds for the agent image, on both tracks
- Warm p95: a few seconds for a routed ticket, dominated by the model call
- AWS forces a cold start when you publish a version and move the alias `live`; GCP when a new revision serves
- Both numbers go in `NOTES.md` before you decide anything

A rollback you have never watched is a plan, not a rollback

AWS. Set `NW_AGENT_ROLE=broken`, then `scripts/deploy_aws.sh rollout agent`.

CodeDeploy moves the alias `live` 10 percent of the way; one probe in ten fails; the errors alarm enters ALARM after ten to twelve minutes; CodeDeploy puts the alias back on its own.

You do nothing but watch, and read the deployment history afterwards.

GCP. Same variable on a new revision: the startup probe fails on `/readyz` and it never receives traffic. Then `make deploy-gcp CANARY=10`: the newest revision under `latest` on 10 percent, the previous one under `stable` on 90, two lines on every panel.

Promote with an apply at zero, or `update-traffic --to-tags stable=100`.

One slow configuration on AWS, 10 percent for 15 minutes, because the alarm needs two five-minute periods: a faster shift would finish before the trigger could act. Every later `cdk deploy` takes the same path. Write the deployment id or the traffic command in `NOTES.md`: that is the runbook.

The capstone is two numbers on the board

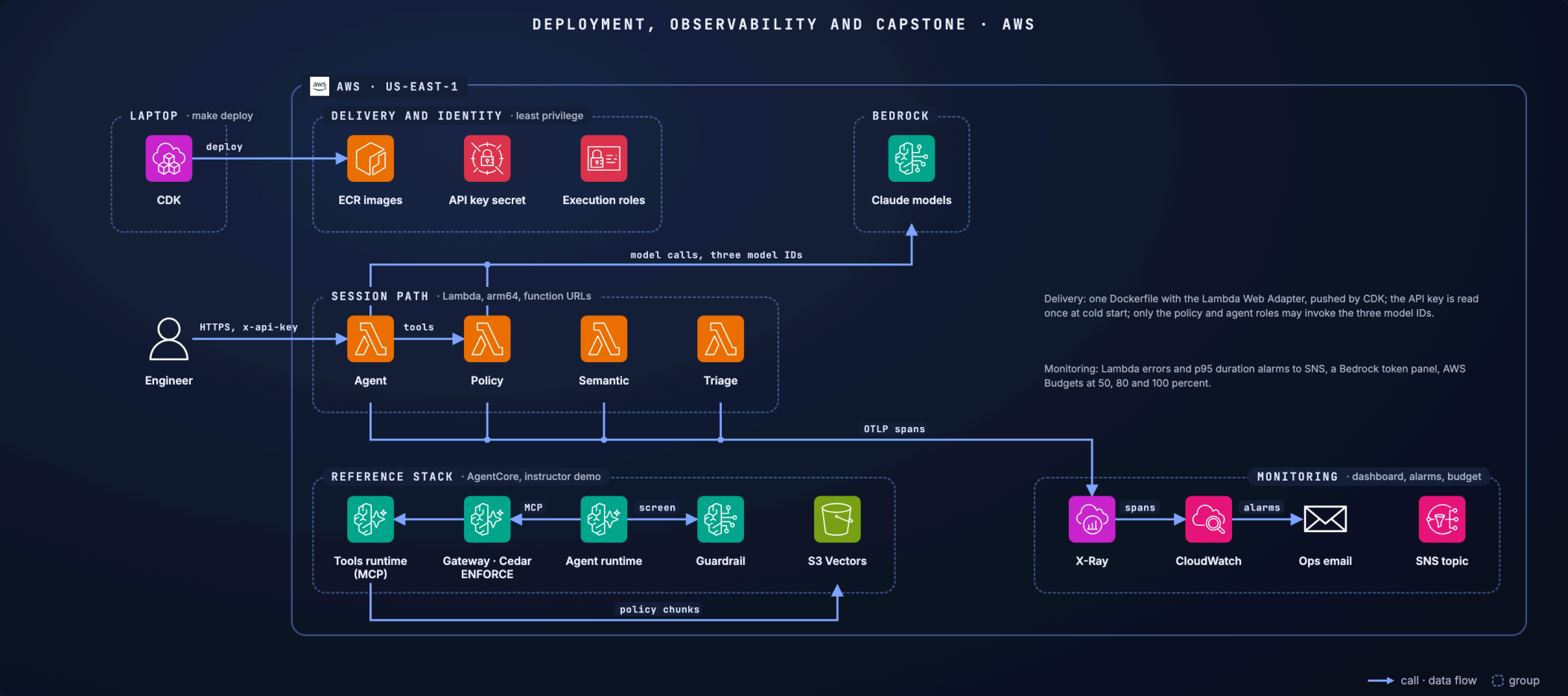
- 1 Twenty tickets through `/run`, the plain loop, four at a time
- 2 The same twenty through `/route`, your router in front
- 3 Cost per resolution for each, and the saving as a percentage
- 4 At Northwind's eight thousand tickets a quarter: what does each architecture cost, and who signs off on the difference?

**about 3
USD**

forty agent runs at up to a minute each, about five minutes wall clock

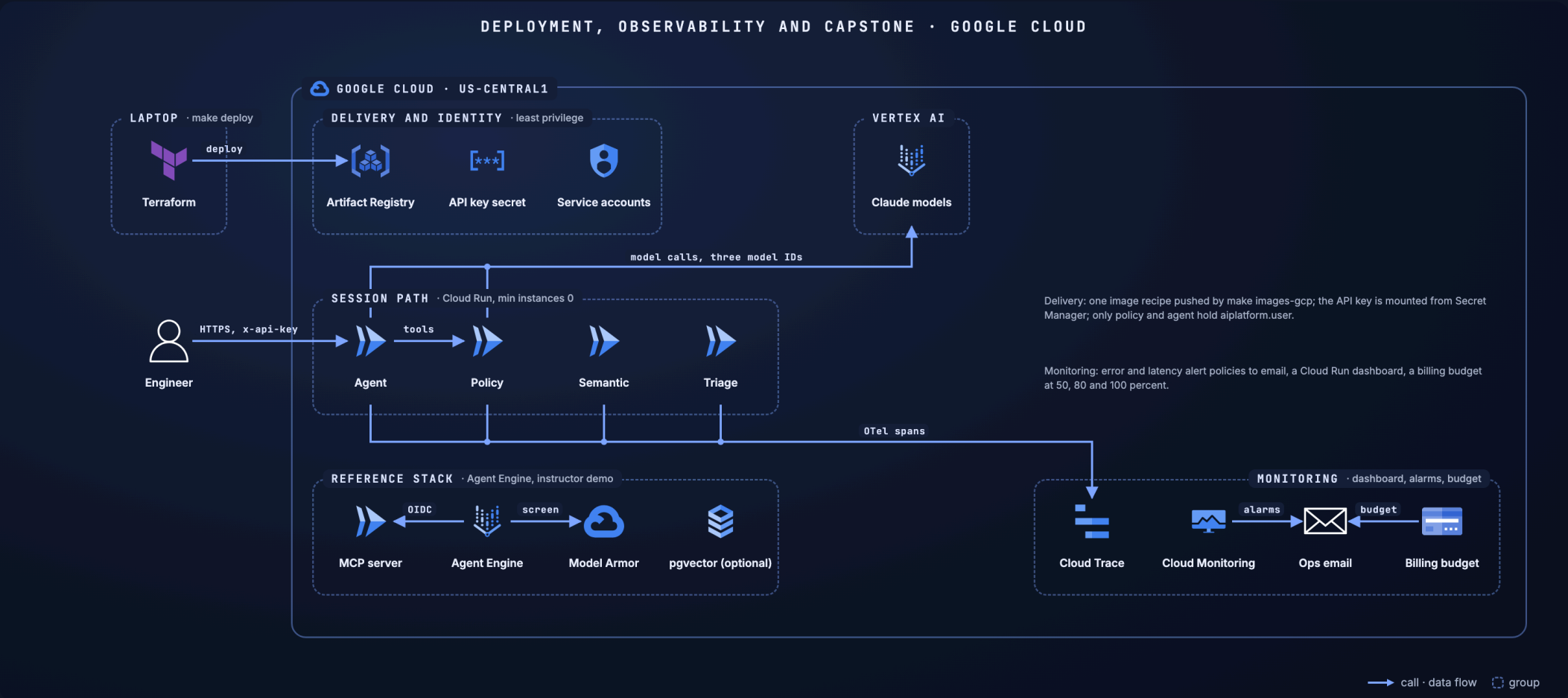
Argue with the number before you argue with the architecture.

What exists after today, AWS track



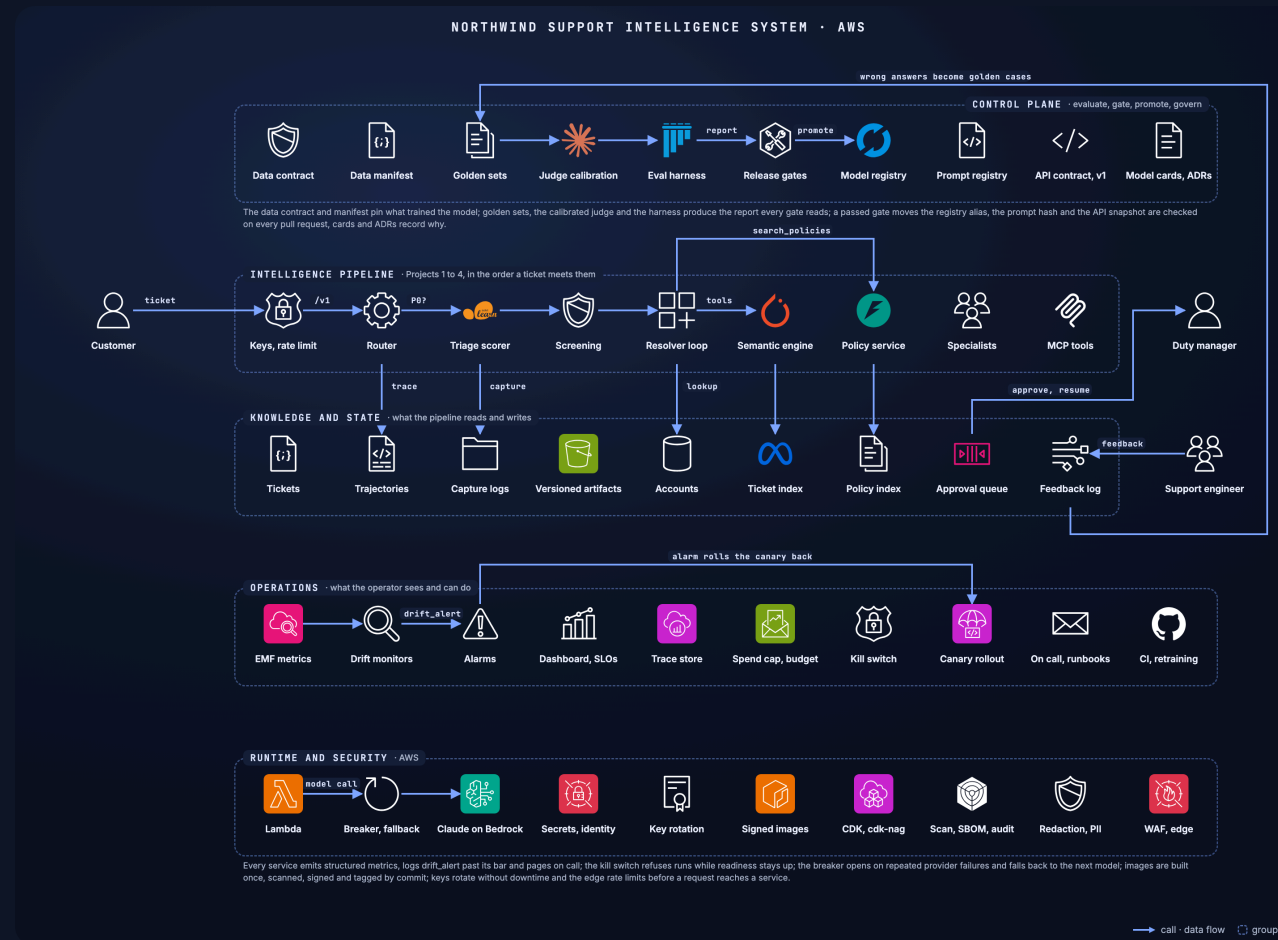
The Session path: four Lambda functions on arm64 images behind function URLs, one execution role each, the API key in Secrets Manager, X-Ray, a CloudWatch dashboard, alarms and a budget. Ringed: the Reference stack the instructor shows.

What exists after today, Google Cloud track



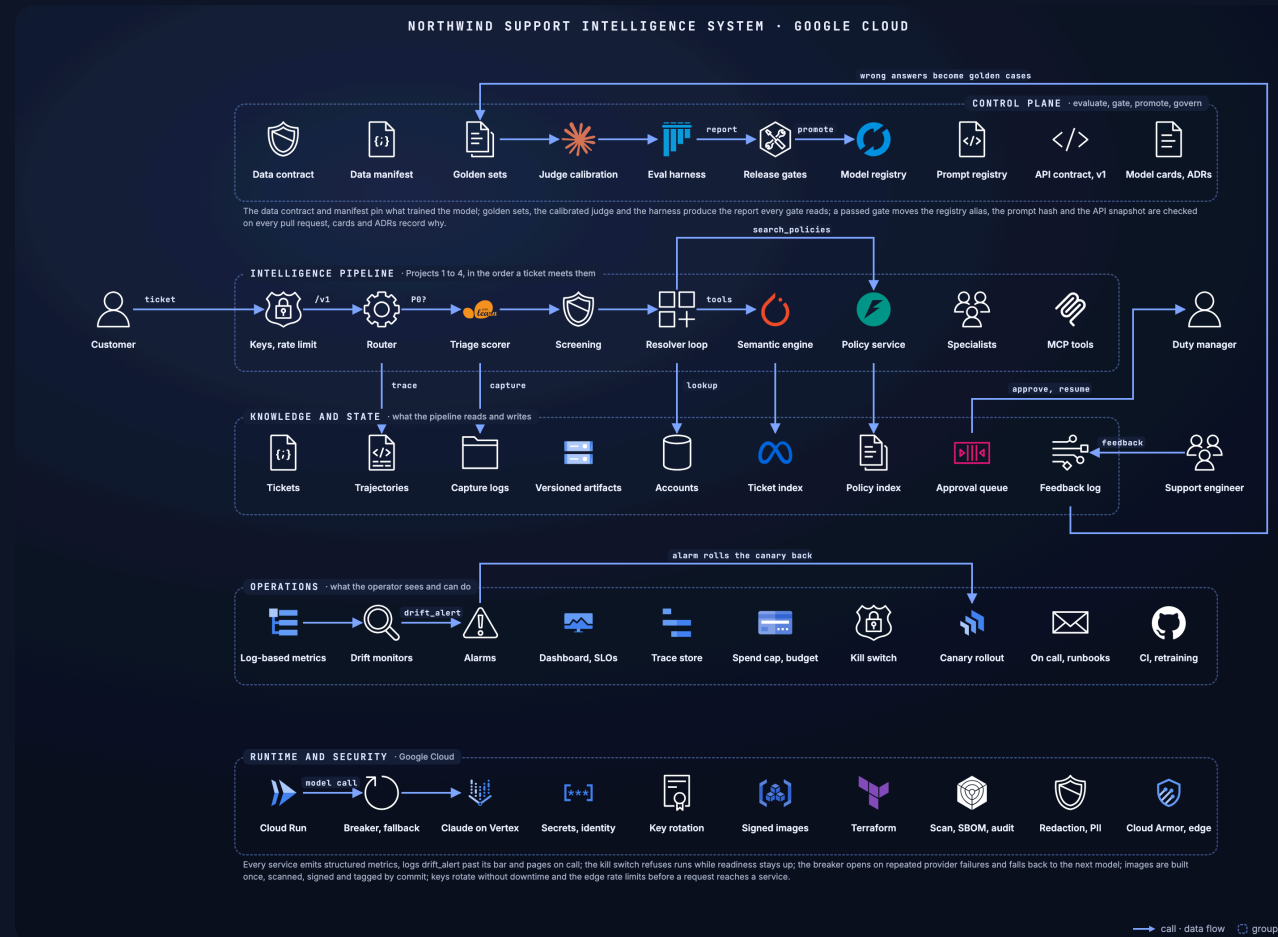
The Session path: five Cloud Run services with one service account each, the API key in Secret Manager, Cloud Trace, a Cloud Monitoring dashboard, alert policies and a billing budget. Ringed: the Reference stack.

The whole system, AWS track



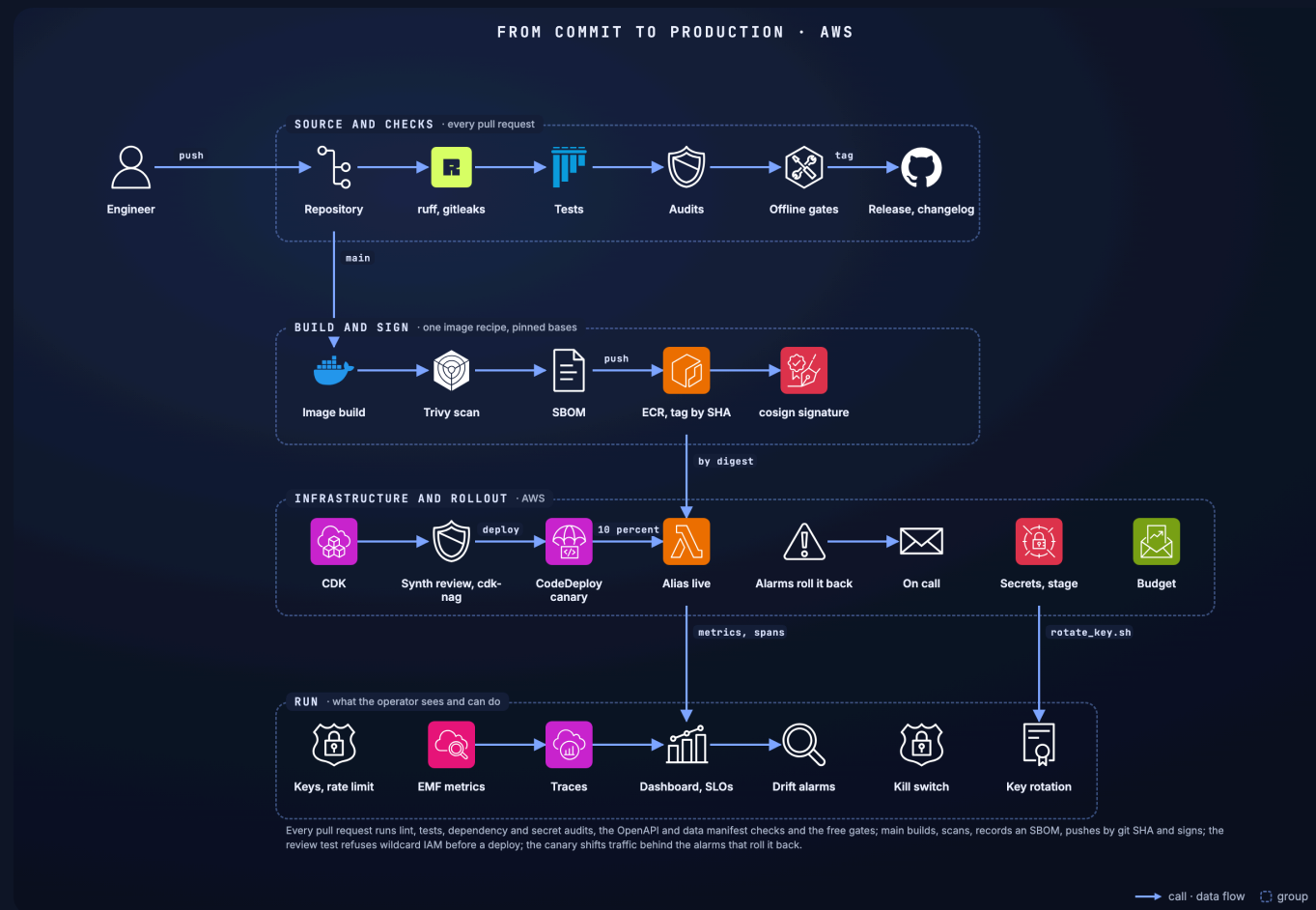
Five rows: the control plane that evaluates, gates, promotes and governs; a ticket's path through the four projects behind keys and a rate limit; what they read and write; the operations that watch and roll back; the runtime and security underneath. Two loops people close: approvals and feedback.

The whole system, Google Cloud track



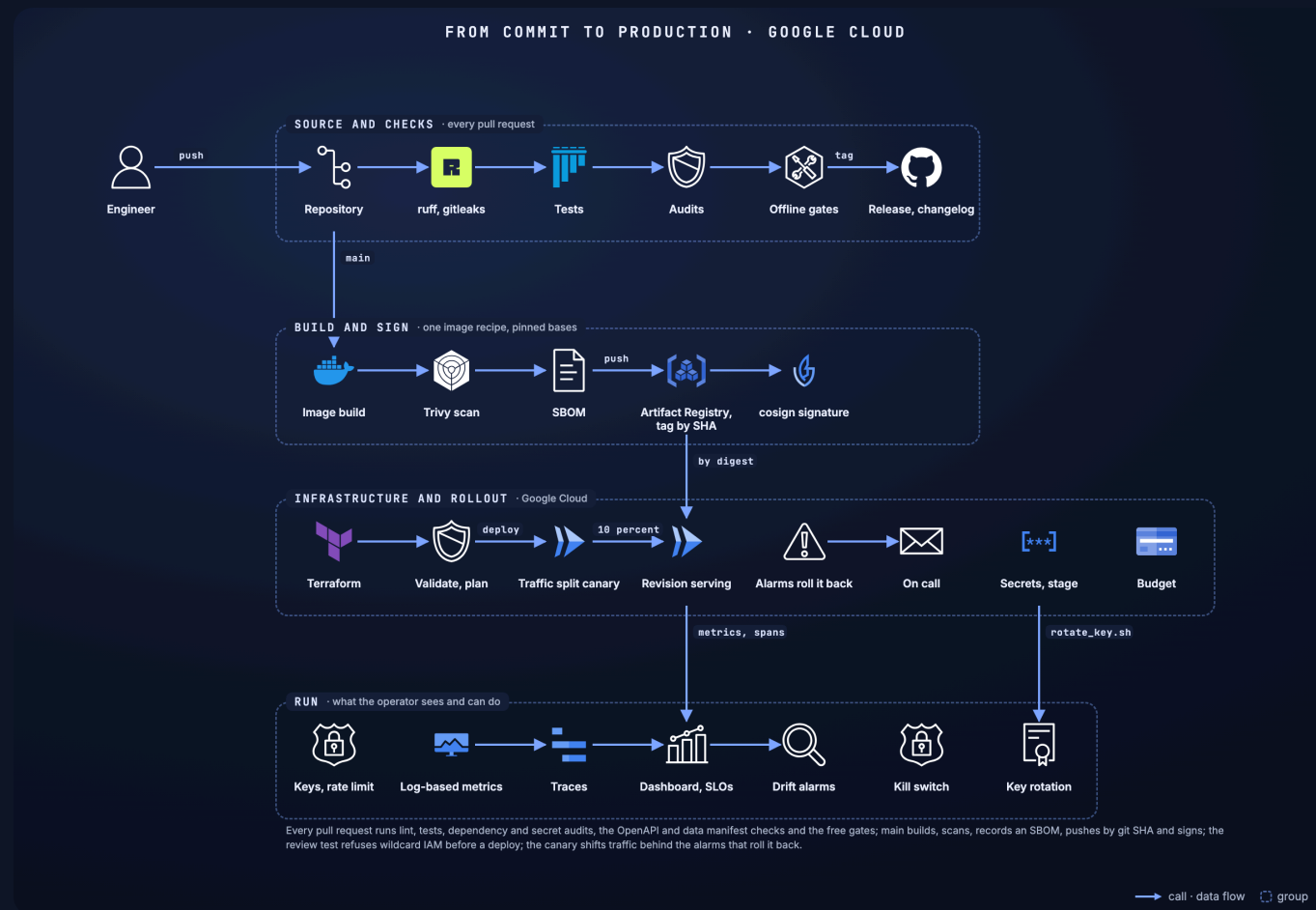
The same system with Cloud Run, Vertex AI, Cloud Trace, log-based metrics and Cloud Armor underneath it; nothing above the runtime row changes between tracks.

From commit to production, AWS track



Every pull request runs lint, tests, audits and the offline gates; main builds one image, scans it, writes the SBOM, signs it and tags it by commit; CDK is synthesised and reviewed before CodeDeploy shifts the live alias ten percent at a time behind the alarms.

From commit to production, Google Cloud track



The same checks and the same image recipe; Terraform is validated and planned, then a Cloud Run traffic split puts the new revision on ten percent while the log-based metrics and the alert policies decide whether it stays.

The Reference stack puts the agent on the managed runtime

	AWS	GCP
Agent runtime	two AgentCore runtimes on the <code>nw-agent</code> image: the resolver and the tool registry as an MCP server	the resolver on Vertex AI Agent Engine, same image, calling the Session path services as HTTP tools
Tool access	an AgentCore gateway with a Cedar policy engine in ENFORCE mode	the MCP server as a private Cloud Run service, invokable only by the agent identity
Guardrail	a Bedrock guardrail in front of the loop	a Model Armor template in front of the loop
Retrieval	S3 Vectors, filled after the deploy for the demo	pgvector on Cloud SQL, optional and off by default
Per day of demos	about 1 USD, most of it the guardrail	under 10 USD per month; Cloud SQL adds about 50 USD per month

The platform enforces the escalation gate a second time

AWS

The Cedar policy permits the six read tools to any IAM caller and forbids `northwind-tools__escalate` unless the caller is an assumed role under `NorthwindApprovers`. The runtime's own role is not, so the platform denies it.

Bedrock Guardrails: prompt attack at low strength, PII anonymised, grounding checked.

GCP

The MCP service accepts calls only from the agent identity, and Model Armor screens the prompt before the loop runs. The loop's own gate still decides whether `escalate` executes.

Model Armor: prompt injection and jailbreak at medium confidence, sensitive data, malicious URLs. Inspect and block.

Low strength is deliberate. Argus measured that high strength blocks about three percent of ordinary support prompts, because tool results are machine-built text. Watch `InvocationsIntervened` before tightening anything.

Acceptance criteria

1. A public HTTPS endpoint answers a real ticket end to end
2. No credentials in the deployed image, verified by reading its layers
3. Cold-start and warm p95 latency both measured and written down
4. A dashboard whose request and latency panels move with your tickets, token spend per model on AWS, and a budget alarm that exists
5. A rollback command you have actually run once

45 and 18 passed

the two lines `make session06` prints: the router and the runtime contract, then on AWS the synth review with the alias, CodeDeploy and stage checks

Questions worth asking

- Your Session path costs about three cents a day idle on AWS and nothing on GCP. Which one did you expect, and why is it that way?
- What is your cost per resolved ticket, and what would Northwind accept at eight thousand tickets a quarter?
- Of every failure you saw across the six parts, which would have reached production in your current setup?
- Which component do you replace first when the model vendor changes, and what in this repo would you touch?